Purple Team in AWS using Stratus Red Team

This document outlines the steps for setting up an AWS environment and using Stratus Red Team to run a purple team exercise. The process involves creating infrastructure with Terraform, configuring instances, simulating attacker techniques, and validating that the resulting activity is visible in CloudTrail so detection controls can be tested and tuned.

Index

- Prerequisites
- Step 1: Configure AWS Credentials
- Step 2: Create Your Terraform Project Directory
- Step 3: Create the Terraform Configuration Files
- Step 4: Generate SSH Key Pair (Windows)
- Step 5: Run Terraform Commands
- Step 6: Access Your Instances
- Step 7: Enable AWS Systems Manager
- Step 8: Install on Linux
- Step 9: Install on Windows
- Step 10: Clean Up (Destroy Resources)
- Outcome

Prerequisites

Before beginning, you'll need the following on a Windows machine:

- **Chocolatey (choco):** A package manager for Windows, installed with a PowerShell command that bypasses the execution policy for the current process.
- **Terraform:** Installed with `choco install terraform` in PowerShell.
- **AWS CLI:** Not strictly required, but good practice.
- **AWS Account:** An active AWS account.
- **AWS IAM User:** An IAM user with programmatic access and permissions to create VPCs, subnets and EC2 instances. AmazonEC2FullAccess and VPCFullAccess are a good starting point. Save the Access key ID and Secret access key.

Installation of Prerequisites

1 Install choco:

```
Set-ExecutionPolicy Bypass -Scope Process -Force; [System.Net.ServicePointManager]::SecurityProtocol = [S \
    ystem.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-Object System.Net.WebClient).Do \
    wnloadString('https://community.chocolatey.org/install.ps1'))
```

2 Install Terraform:

```
choco install terraform
```

3 Install the AWS CLI: https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2-windows.html

4 Make sure you have an AWS account and an IAM user as described above.

Step 1: Configure AWS Credentials

The most common and secure way to provide AWS credentials to Terraform is with environment variables. Open a Command Prompt or PowerShell terminal and set the following, replacing the placeholders with your IAM user credentials:

```
set AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY_ID"
set AWS_SECRET_ACCESS_KEY="YOUR_SECRET_ACCESS_KEY"
set AWS_DEFAULT_REGION="us-east-1" # Or your desired AWS region
```

Step 2: Create Your Terraform Project Directory

```
mkdir terraform-aws-ec2
cd terraform-aws-ec2
```

Step 3: Create the Terraform Configuration Files

We will create a few files to keep the configuration organized.

main.tf

This file contains the core resources: the VPC, subnets, security group and the EC2 instances.

```
# main.tf

# Configure the AWS Provider
provider "aws" {
  region = var.aws_region
}

# 1. Create a VPC
resource "aws_vpc" "main_vpc" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "MyVPC"
  }
}

# 2. Create two Subnets
resource "aws_subnet" "public_subnet_1" {
  vpc_id                  = aws_vpc.main_vpc.id
  cidr_block              = var.subnet_cidr_1
  availability_zone       = var.aws_region_az_1
  map_public_ip_on_launch = true # To get a public IP for instances
  tags = {
    Name = "PublicSubnet1"
  }
}

resource "aws_subnet" "public_subnet_2" {
  vpc_id                  = aws_vpc.main_vpc.id
  cidr_block              = var.subnet_cidr_2
  availability_zone       = var.aws_region_az_2
  map_public_ip_on_launch = true
  tags = {
    Name = "PublicSubnet2"
  }
```

```
}

# 3. Create an Internet Gateway and attach it to the VPC
resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main_vpc.id
  tags = {
    Name = "MyInternetGateway"
  }
}

# 4. Create a Route Table and a Route to the Internet
resource "aws_route_table" "rt" {
  vpc_id = aws_vpc.main_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.gw.id
  }
  tags = {
    Name = "MyRouteTable"
  }
}

# 5. Associate the Route Table with the Subnets
resource "aws_route_table_association" "a_1" {
  subnet_id      = aws_subnet.public_subnet_1.id
  route_table_id = aws_route_table.rt.id
}

resource "aws_route_table_association" "a_2" {
  subnet_id      = aws_subnet.public_subnet_2.id
  route_table_id = aws_route_table.rt.id
}

# 6. Create a Security Group
resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow inbound traffic for web and SSH"
  vpc_id      = aws_vpc.main_vpc.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "RDP from anywhere"
    from_port   = 3389
    to_port     = 3389
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  tags = {
    Name = "web-sg"
  }
}

# 7. Create a Key Pair for Linux
resource "aws_key_pair" "keypair" {
  key_name   = "my-linux-key"
  public_key = file(var.public_key_path)
}

# 8. Create a Linux EC2 Instance
resource "aws_instance" "linux_instance" {
  ami                         = data.aws_ami.linux_ami.id
  instance_type               = var.linux_instance_type
  subnet_id                   = aws_subnet.public_subnet_1.id
  vpc_security_group_ids      = [aws_security_group.web_sg.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.keypair.key_name
  tags = {
    Name = "LinuxInstance"
  }
}

# 9. Create a Windows EC2 Instance
resource "aws_instance" "windows_instance" {
  ami                         = data.aws_ami.windows_ami.id
  instance_type               = var.windows_instance_type
  subnet_id                   = aws_subnet.public_subnet_2.id
  vpc_security_group_ids      = [aws_security_group.web_sg.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.keypair.key_name
  tags = {
    Name = "WindowsInstance"
  }
}

# Get the latest AMI for Amazon Linux
data "aws_ami" "linux_ami" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Get the latest AMI for Windows Server 2022
data "aws_ami" "windows_ami" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["Windows_Server-2022-English-Full-Base-*"]
  }
}
```

variables.tf

```
# variables.tf

variable "aws_region" {
```

```
  description = "The AWS region to create resources in."
  type        = string
  default     = "us-east-1"
}

variable "aws_region_az_1" {
  description = "The first availability zone."
  type        = string
  default     = "us-east-1a"
}

variable "aws_region_az_2" {
  description = "The second availability zone."
  type        = string
  default     = "us-east-1b"
}

variable "vpc_cidr" {
  description = "The CIDR block for the VPC."
  type        = string
  default     = "10.0.0.0/16"
}

variable "subnet_cidr_1" {
  description = "The CIDR block for the first subnet."
  type        = string
  default     = "10.0.1.0/24"
}

variable "subnet_cidr_2" {
  description = "The CIDR block for the second subnet."
  type        = string
  default     = "10.0.2.0/24"
}

variable "linux_instance_type" {
  description = "The instance type for the Linux EC2 instance."
  type        = string
  default     = "t2.medium"
}

variable "windows_instance_type" {
  description = "The instance type for the Windows EC2 instance."
  type        = string
  default     = "t2.medium"
}

variable "public_key_path" {
  description = "Path to the public SSH key for the Linux instance."
  type        = string
  default     = "~/.ssh/id_rsa.pub" # Adjust this path as needed for your Windows machine
}
```

Note on public_key_path: On Windows, your SSH keys may be in a different location. A common one is `C:\Users\YourUsername\.ssh\id_rsa.pub`. Generate an SSH key pair if you don't have one, using Git Bash or PuTTYgen.

outputs.tf

```
# outputs.tf

output "linux_instance_public_ip" {
  description = "Public IP address of the Linux instance."
  value       = aws_instance.linux_instance.public_ip
}
```

```
output "windows_instance_public_ip" {
  description = "Public IP address of the Windows instance."
  value       = aws_instance.windows_instance.public_ip
}

output "vpc_id" {
  description = "ID of the created VPC."
  value       = aws_vpc.main_vpc.id
}
```

Step 4: Generate SSH Key Pair (Windows)

You need an SSH key pair to access the Linux instance.

1. **Open Git Bash or Command Prompt.** Git Bash is easiest because it includes `ssh-keygen`.
2. **Generate a key pair:**

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
```

3. Press Enter to accept the default file name and location. You can leave the passphrase empty for this example.
4. **Verify:** The command creates `id_rsa` (private key) and `id_rsa.pub` (public key) in `C:\Users\YourUsername\.ssh`.

Make sure the `public_key_path` variable in `variables.tf` points to this file.

Step 5: Run Terraform Commands

Navigate to your project directory (`terraform-aws-ec2`) in Command Prompt or PowerShell.

1. **Initialize Terraform** (downloads the AWS provider plugin):

```
terraform init
```

2. **Review the plan** (shows what Terraform will create, modify or destroy):

```
terraform plan
```

You should see a plan to add 1 VPC, 2 subnets, 2 instances and other related resources.

3. **Apply the configuration:**

```
terraform apply
```

Terraform shows the plan again and asks for confirmation. Type `yes` and press Enter. Provisioning can take a few minutes, especially for the Windows instance.

4. **Check the outputs.** When the apply completes, the public IPs appear in the output:

```
Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

linux_instance_public_ip = "203.0.113.10"
windows_instance_public_ip = "203.0.113.20"
vpc_id = "vpc-0xxxxxxxxxxxxxxx"
```

Step 6: Access Your Instances

Linux instance: Use an SSH client (such as Git Bash) with the private key you generated.

```
ssh -i "C:\Users\YourUsername\.ssh\id_rsa" ec2-user@<linux_instance_public_ip>
```

Windows instance:

1. Go to the AWS Management Console -> EC2 -> Instances.
2. Select your Windows instance.
3. Click "Connect".
4. Select the "RDP client" tab.
5. Click "Get password".
6. Upload the `id_rsa` private key file you generated when prompted.
7. Click "Decrypt Password".
8. Download the RDP file and use the decrypted password to connect to the Windows machine.

Final result

[Screenshot: Terraform apply output, resource IDs blurred]

Apply command successful.

Public instances:

- Linux -> 203.0.113.10 (the IP will change)
- Windows -> 203.0.113.20 (the IP will change)

[Screenshot: Apply complete, IPs and VPC ID blurred]

Validate the instances in the respective region:
https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Overview:

[Screenshot: EC2 instances list, instance IDs blurred]

Step 7: Enable AWS Systems Manager

To use SSM, attach an IAM role with the `AmazonSSMManagedInstanceCore` policy to your EC2 instance. In the AWS Console, go to EC2 > Actions > Security > Modify IAM role and attach the role.

Step 8: Install on Linux

Connect to the instance:

```
ssh -i "C:\Users\YourUsername\.ssh\id_rsa" ec2-user@<linux_instance_public_ip>
```

Install Stratus Red Team (pre-built binary):

```
wget https://github.com/DataDog/stratus-red-team/releases/download/v2.23.2/stratus-red-team_Linux_x86_64. \
    tar.gz
tar -xvf stratus-red-team_Linux_x86_64.tar.gz
sudo mv stratus /usr/local/bin/
```

Verify:

```
stratus version
```

[Screenshot: stratus version and stratus list output]

Configure Static Credentials

Create the credentials file:

```
mkdir -p ~/.aws
nano ~/.aws/credentials
```

Add:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY_ID
aws_secret_access_key = YOUR_SECRET_ACCESS_KEY
```

Create the config file:

```
nano ~/.aws/config
```

Add:

```
[default]
region = YOUR_REGION
```

Optionally, set the values as environment variables:

```
nano ~/.bashrc
# add the export lines (VALUES)

nano ~/.bash_profile
# add the export lines (VALUES)

source ~/.bashrc
```

Test 1: Console Login without MFA

Technique reference:
https://stratus-red-team.cloud/attack-techniques/AWS/aws.initial-access.console-login-without-mfa/

```
stratus warmup aws.initial-access.console-login-without-mfa
```

[Screenshot: warmup output, account ID, password and sign-in link blurred]

```
stratus detonate aws.initial-access.console-login-without-mfa
```

[Screenshot: detonate output, account ID blurred]

[Screenshot: IAM users list]

User created: `stratus-red-team-xxxxx-xxxxxxxxxx`

```
stratus cleanup aws.initial-access.console-login-without-mfa
```

[Screenshot: cleanup output]

[Screenshot: IAM users after cleanup]

CloudTrail monitoring

[Screenshot: CloudTrail events CreateLoginProfile, CreateUser, AttachUserPolicy]

Step 9: Install on Windows

Add S3FullAccess to the 2 users.

Verify connectivity using SSM Session Manager.

[Screenshot: SSM Session Manager PowerShell session, instance and session IDs blurred]

Install Stratus Red Team (pre-built binary):

```
Invoke-WebRequest -Uri 'https://github.com/DataDog/stratus-red-team/releases/download/v2.23.2/stratus-red \
    -team_Windows_x86_64.tar.gz' -OutFile 'stratus-red-team_Windows_x86_64.tar.gz'
```

Extract:

```
tar -xzvf 'stratus-red-team_Windows_x86_64.tar.gz'
```

Verify:

```
stratus.exe
```

[Screenshot: stratus.exe usage output]

Set the environment variables and region in PowerShell:

```
$env:AWS_REGION = "us-east-1"
$env:AWS_ACCESS_KEY_ID = "YOUR_ACCESS_KEY_ID"
$env:AWS_SECRET_ACCESS_KEY = "YOUR_SECRET_ACCESS_KEY"
```

Test 1: Console Login without MFA

Technique reference:
https://stratus-red-team.cloud/attack-techniques/AWS/aws.initial-access.console-login-without-mfa/

```
stratus warmup aws.initial-access.console-login-without-mfa
```

[Screenshot: warmup output]

```
stratus detonate aws.initial-access.console-login-without-mfa
```

[Screenshot: detonate output, account ID blurred]

[Screenshot: IAM users list]

User created: `stratus-red-team-xxxxx-xxxxxxxxxx`

```
stratus cleanup aws.initial-access.console-login-without-mfa
```

[Screenshot: cleanup output]

CloudTrail monitoring

[Screenshot: CloudTrail events ConsoleLogin, CreateLoginProfile, CreateUser]

Step 10: Clean Up (Destroy Resources)

When you're done, destroy the resources to avoid unnecessary AWS costs.

```
terraform destroy
```

Terraform shows the resources it plans to destroy. Type `yes` to confirm. This terminates the EC2 instances and deletes all the resources created in your VPC.

Outcome

The final result of the process is ***the successful creation of the AWS resources and the installation and verification of Stratus Red Team on both a Linux and a Windows instance.*** The `terraform apply` command adds 11 resources without any changes or destructions. The outputs show the public IP addresses of

the Linux and Windows instances and the VPC ID.

After installation, the `stratus warmup` and `stratus detonate` commands for the `aws.initial-access.console-login-without-mfa` technique are executed, which successfully perform a console login for a newly created IAM user. The `stratus cleanup` command is then used to remove the resources associated with the technique.

CloudTrail recorded `CreateUser`, `CreateLoginProfile` and `ConsoleLogin`, confirming that these events can be used to build and validate detection rules.

Created by Ju4nj0